YSSE Alumni Directory: Project & Technical Documentation

Internship Deliverable: YSSE Alumni Directory, initial demo version

Author: Syed Ziaul Hasan

Date: October 2026

Architecture: HTML + CSS + JavaScript (client) | Node.js + Express + SQLite (server)

Table of Contents

1. Project Overview
2. Features and Functionalities Implemented
3. Technologies, Frameworks, and Libraries
4. Screenshots of Main Pages
5. Project Structure and Implementation Overview
6. Setup and Installation Instructions
7. Completed Features, Pending Work, and Issues Faced
8. Testing and Security Summary
9. Known Limitations and Future Work
10. Appendix A: Configuration Reference
11. Appendix B: API Endpoint Reference
12. Appendix C: Executed Verification Commands and Outputs

1. Project Overview

1.1 Purpose

The YSSE Alumni Directory is a centralized website where YSSE alumni profiles can be explored, searched and filtered, where alumni professional journeys and achievements are showcased, and where current members can find alumni for networking, knowledge sharing and mentorship. This version is the initial demo: it establishes the structure, the main pages and the first working features, using sample alumni data.

1.2 Target Audiences

1. **Visitors and current members:** browse the directory, read profiles, explore Alumni Talk sessions and achievements, and send a connection request to an alumnus.
2. **Alumni:** review and update their own profile (role, workplace, bio, mentoring availability) and submit the changes for approval.
3. **Admin team:** review submitted profiles and updates, approve or reject them, and manage Alumni Talk entries.

1.3 Scope of This Version

- Seven pages: Home, Directory, Alumni Profile, Alumni Talk, Achievements, My profile (user dashboard) and Admin dashboard.
- A SQLite database behind a small Express API, filled with sample data on first run.
- An offline demo mode that runs from `index.html` alone, using the same sample data.
- Sample data: 8 alumni, 4 talks, 4 achievements and 2 pending profile submissions.

2. Features and Functionalities Implemented

2.1 Grouped Feature Description

A. Homepage

- Hero section with the headline, a short introduction and a search box. The search text is passed to the directory page.
- Live alumni count taken from the loaded data.
- Three featured alumni cards.
- Shortcut buttons to the directory, Alumni Talk and achievements pages.

Key files: `js/pages.js` (`Pages[""]`), `css/styles.css`.

B. Alumni Directory and Search

- Card for every alumnus: initials avatar, name, role and company, batch, department and an "Open to mentoring" tag.
- Search box matching name, company, role or batch.
- Department filter (Engineering, Business, Science, Arts & Humanities).
- Batch filter built from the batches present in the data.
- "Open to mentoring" checkbox.
- Results update on every keystroke or change, with a result count and an empty-state message when nothing matches.

Key files: `js/pages.js` (`Pages.directory`, `initDirectory`).

C. Individual Alumni Profiles

- Header with avatar, name, role, workplace, city, batch and department, followed by the bio.
- Professional journey timeline and achievements list.
- Connect panel: LinkedIn link when the alumnus provided one, otherwise a note that contact details are private.
- "Send connection request" form with a required message field; submitting shows a confirmation message.
- A friendly "not found" message for unknown profile links.

Key files: `js/pages.js` (`Pages.alumni`), `js/app.js` (connect form handler).

D. Alumni Talk

- Upcoming sessions and past sessions, each with date, title, speaker (linked to the speaker profile) and a "Join live session" or "Watch recording" button.
- Grid of alumni who are open to sharing their experience.

Key files: `js/pages.js` (`Pages.talks`), `js/data.js` (`talks`).

E. Alumni Achievements

- Success-story cards with type (Entrepreneurship, Award, Research, Impact), year, title, description and a link to the alumnus profile.

Key files: `js/pages.js` (`Pages.achievements`), `js/data.js` (`achievements`).

F. User Dashboard (My profile)

- Demo sign-in as one alumnus. Form fields: current role, workplace, short bio and a mentoring/talks checkbox.

- "Submit for approval" sends the change to the admin review queue; a counter shows how many of the alumnus submissions are waiting.

Key files: `js/pages.js` (`Pages.dashboard`), `js/app.js` (profile form handler).

G. Admin Dashboard

- Counters for alumni, profiles to review, talks and achievements.
- Review table listing each submission as a new member or an update (with the changed fields), with Approve and Reject buttons.
- Approving an update changes the alumnus record; approving a new member adds them to the directory.
- Alumni Talk table with a Remove button for each entry.

Key files: `js/pages.js` (`Pages.admin`), `js/app.js` (click handler), `server/server.js`.

H. Database and API Layer

- SQLite database with four tables (`alumni`, `talks`, `achievements`, `pending`), created and filled with sample data on first run.
- Express API that reads all data, stores submissions, approves or rejects them and removes talks.
- Frontend loader (`js/api.js`) that fetches data on page load and sends every change to the server, then reloads the data.
- Automatic demo mode: if the server is not running, the site uses the sample data in `js/data.js` and the footer shows "Demo mode (no server running)". With the server, the footer shows "Connected to database".
- Toast messages confirm each action (submission, approval, rejection, removal, connection request).

2.2 Routes Summary

Route	Page	What it provides
`#/`	Home	Hero search, featured alumni, shortcut buttons
`#/directory`	Directory	Search, department/batch/mentoring filters, alumni cards
`#/alumni/:id`	Alumni profile	Details, journey, achievements, connection request form
`#/talks`	Alumni Talk	Upcoming and past sessions, alumni open to sharing
`#/achievements`	Achievements	Success-story cards
`#/dashboard`	My profile	Edit profile fields and submit for approval
`#/admin`	Admin dashboard	Counters, review queue, talk management

3. Technologies, Frameworks, and Libraries

3.1 Frontend Stack

Technology	Version / Source	Purpose in Application
HTML5	-	Single page shell (`index.html`) with navigation, footer and script tags
CSS3	-	Custom properties for colors and fonts, grid and flexbox layouts, media query for small screens, reduced-motion support
JavaScript	ES2020, no framework	Page rendering functions, hash router, filtering, form handling, server calls with `fetch`
Fraunces	Google Fonts	Heading font (Georgia fallback when offline)

Technology	Version / Source	Purpose in Application
System UI fonts	Browser default	Body text

3.2 Backend Stack (`package.json`)

Package Name	Version	Purpose in Application
Node.js	22 or newer	Runtime for the server (required by better-sqlite3)
express	^5.3.0	HTTP server, JSON API routes and static file serving
better-sqlite3	^13.0.3	SQLite database driver with prepared statements and transactions

3.3 Tools Used During Development

- `npm` for installing dependencies.
- `node` syntax check of the three browser scripts.
- `curl` for exercising the API endpoints.
- Python `reportlab` for generating this PDF.

4. Screenshots of Main Pages

All screenshots were captured in a desktop browser by opening `index.html` directly, so the footer shows demo mode. They show the sample data.

4.1 Homepage (`#/`)

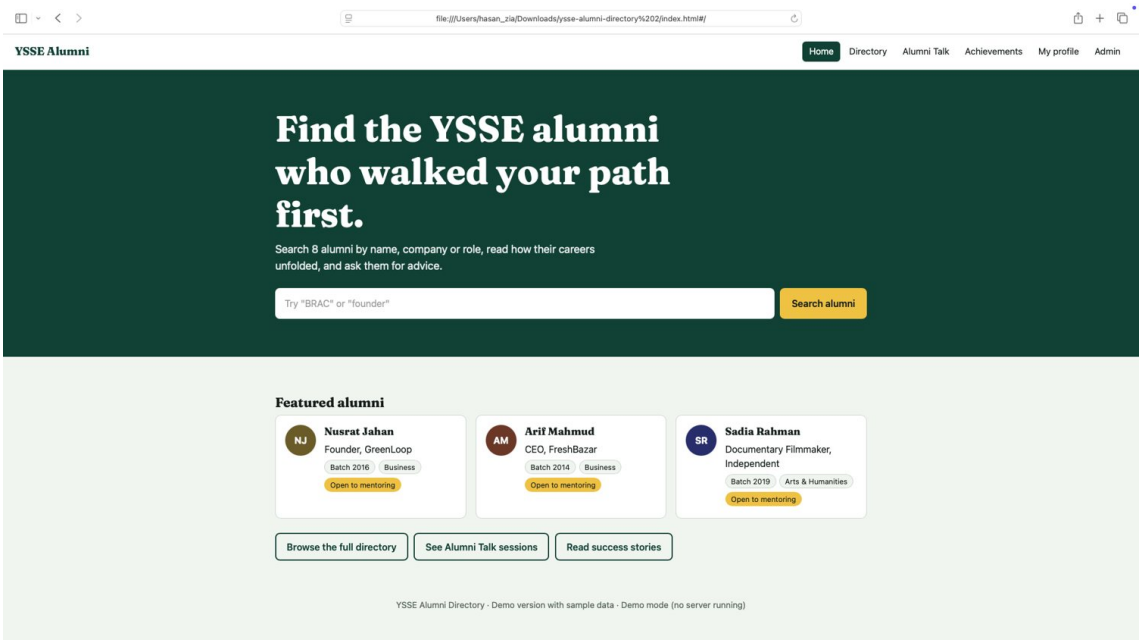


Figure 4.1: Homepage with the hero search box, alumni count, featured alumni and shortcut buttons.

4.2 Alumni Directory (`#/directory`)

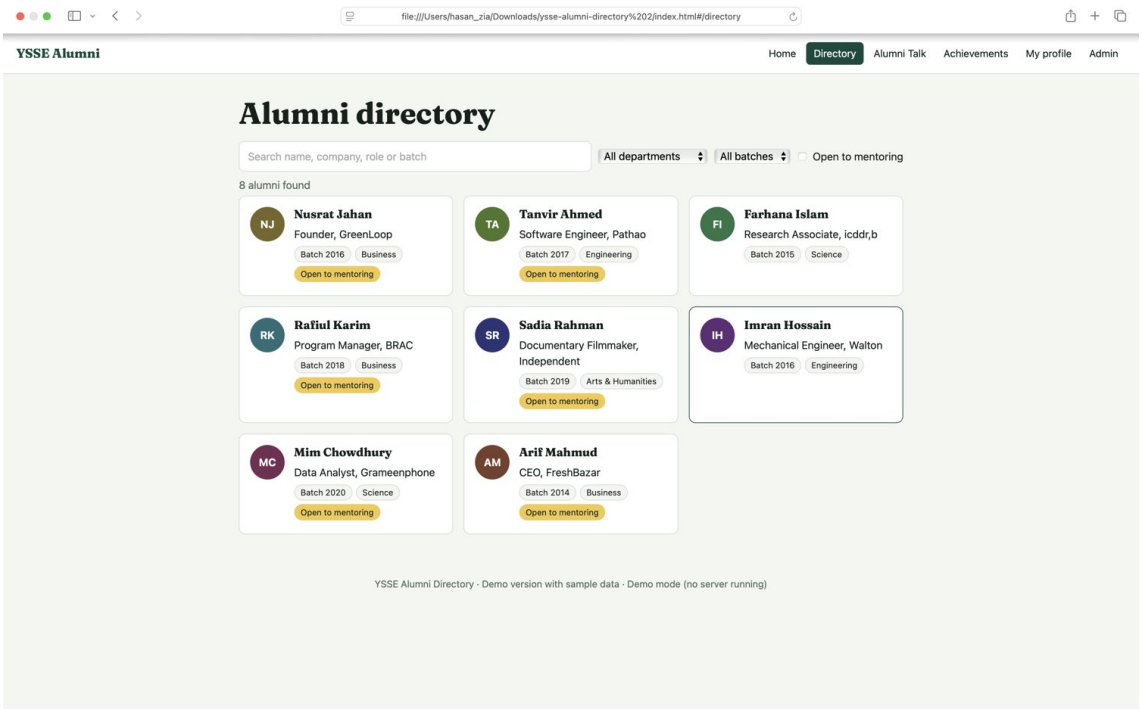


Figure 4.2: Directory with the search box, department and batch filters, mentoring checkbox and alumni cards.

4.3 Alumni Talk (`#/talks`)

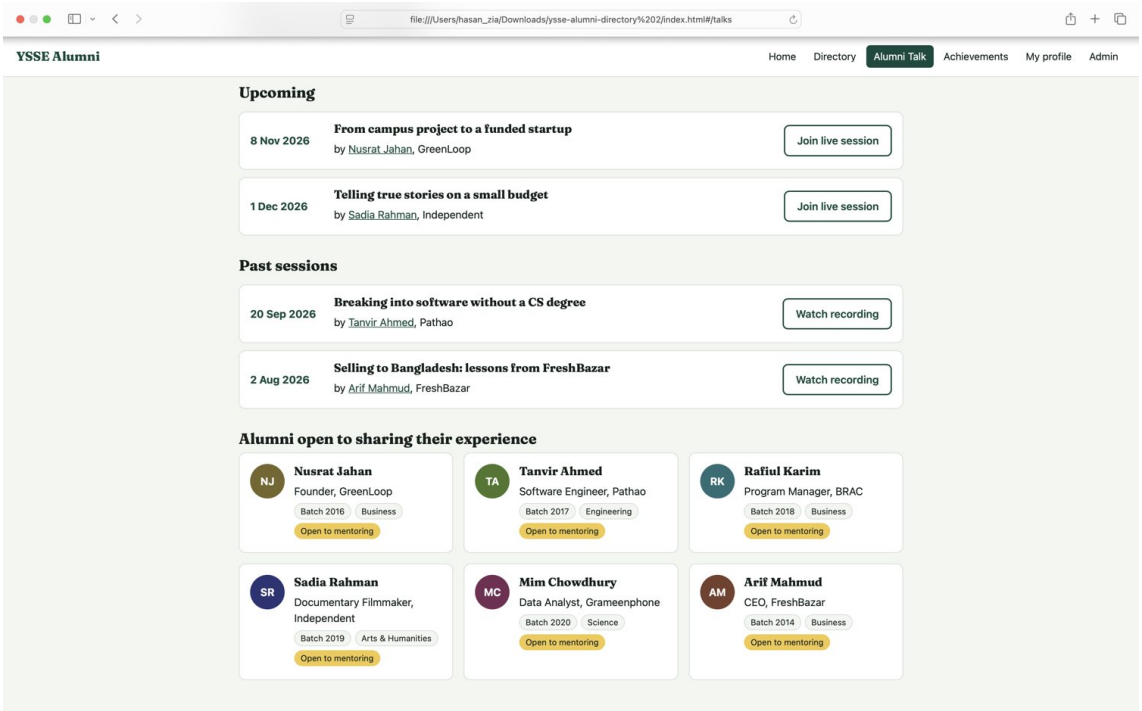


Figure 4.3: Alumni Talk page with upcoming sessions, past sessions and alumni open to sharing.

4.4 Alumni Achievements (`#/achievements`)

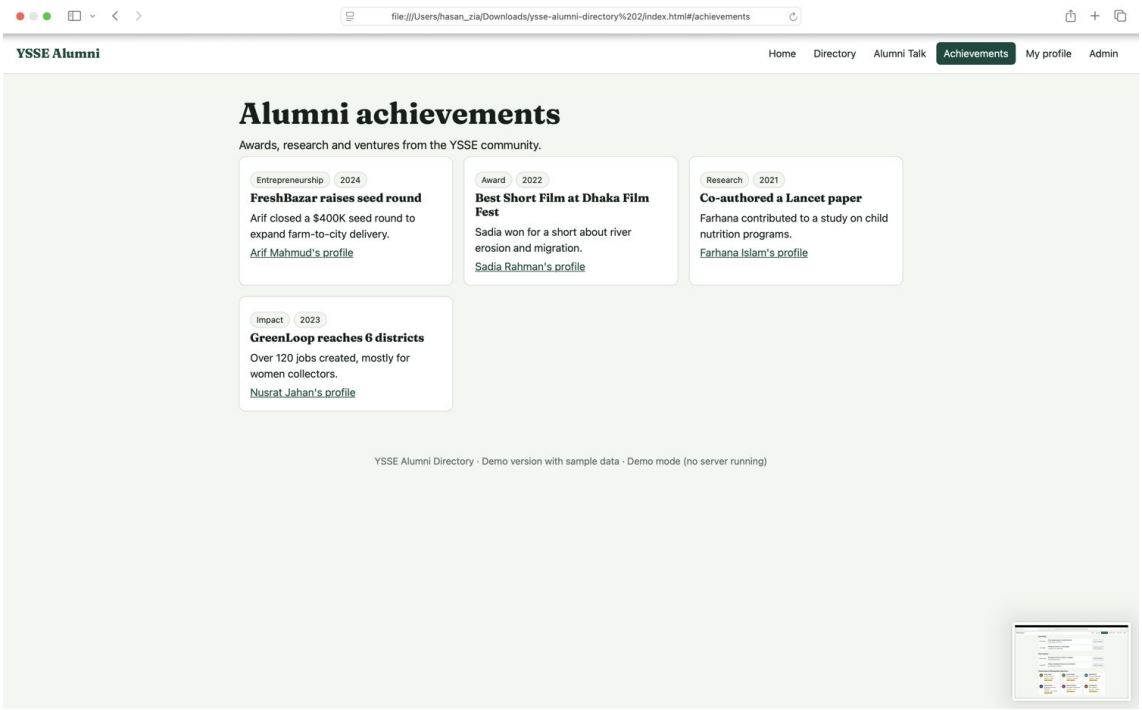


Figure 4.4: Achievements page with success-story cards linked to alumni profiles.

4.5 User Dashboard (`#/dashboard`)

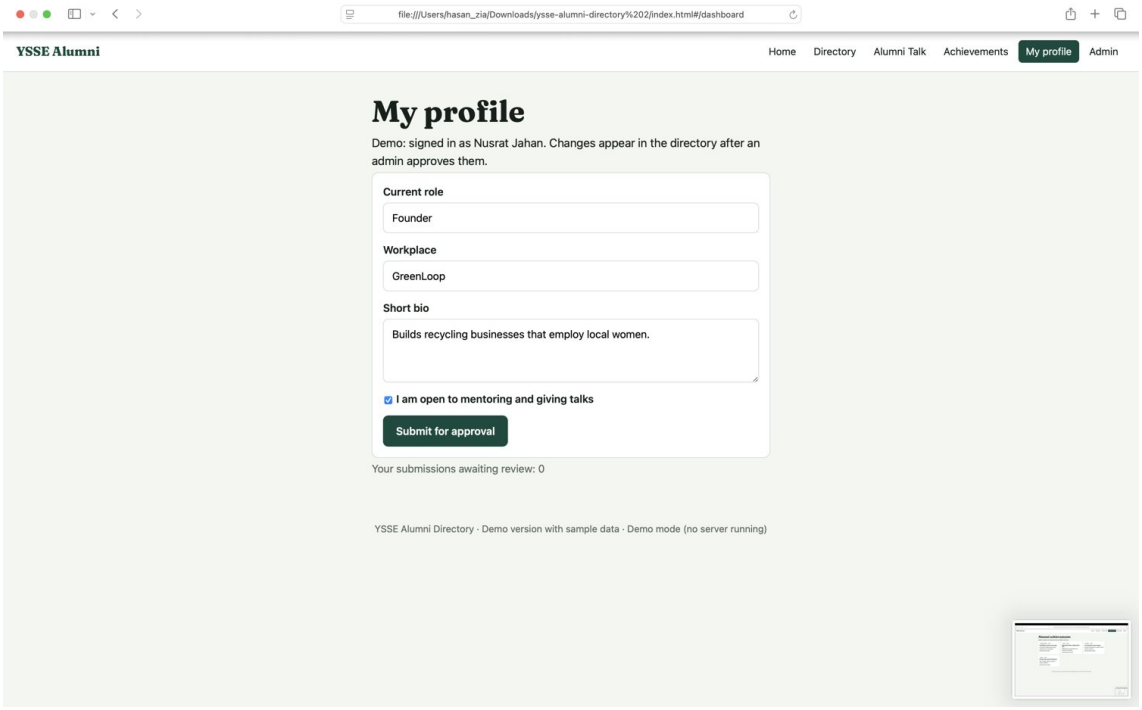


Figure 4.5: My profile page with the editable fields and the Submit for approval button.

4.6 Admin Dashboard (`#/admin`)

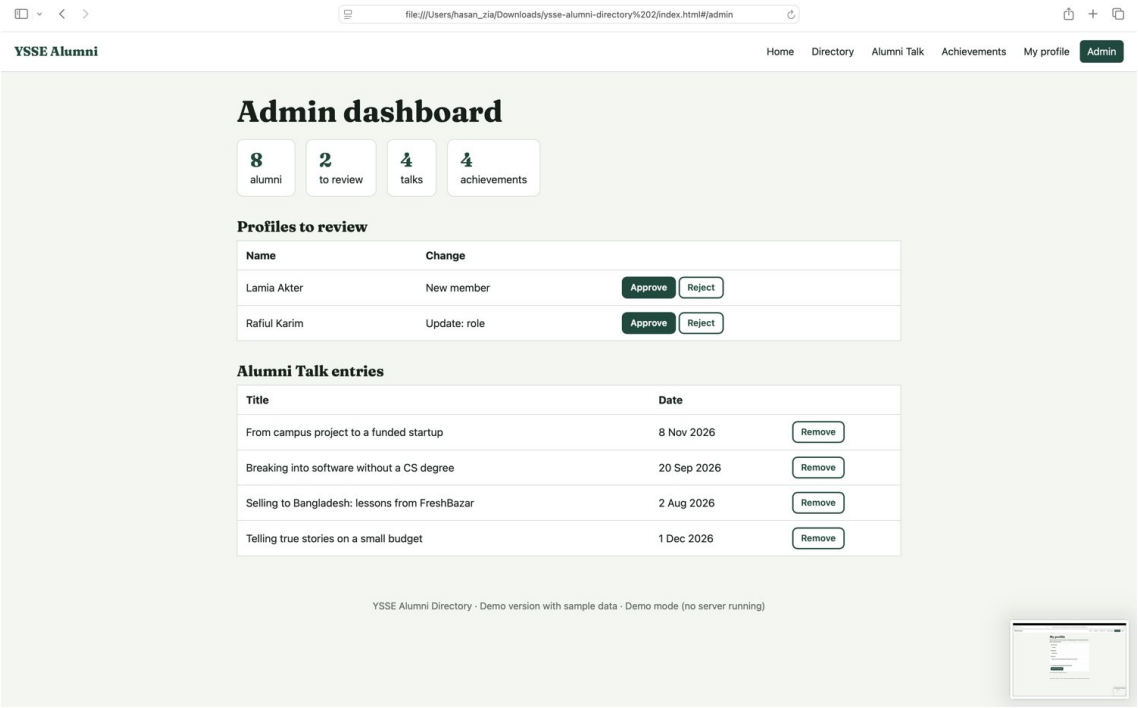


Figure 4.6: Admin dashboard with counters, the profile review table and the Alumni Talk entries table.

5. Project Structure and Implementation Overview

5.1 Annotated Folder Tree

```
ysse-alumni-directory/
|-- index.html            # Page shell: navigation, footer, script tags
|-- package.json          # Dependencies and the npm start script
|-- README.md             # Short run instructions
|-- .gitignore            # Ignores node_modules and database files
|-- css/
|   `-- styles.css        # All styling; colors and fonts are variables at the top
|-- js/
|   |-- data.js           # Sample data (also the database seed)
|   |-- pages.js          # One function per page + directory filter logic
|   |-- api.js            # Loads data from the server, sends changes
|   `-- app.js            # Hash router, click and form handlers
`-- server/
    |-- db.js             # Tables, seeding, writable-field list
    `-- server.js         # Express routes under /api, static files
```

5.2 Data Flow

```
Browser (index.html + js/*)
   |  loadData(): fetch GET /api/data on page load
   |  mutate():   POST / DELETE on every change, then reload data
   v
Express server (server/server.js)
   |  prepared SQL statements and transactions
   v
SQLite file (server/alumni.db)   <- created and seeded on first run
```

If the first `fetch` fails (server not running), `loadData()` marks the site as offline and every `mutate()` call runs a local fallback on the in-memory sample data instead.

5.3 Change Approval Flow

1. An alumnus submits the My profile form. `app.js` calls `mutate()` in `api.js`.
2. `POST /api/pending` stores the submission in the `pending` table.

3. The admin opens the Admin dashboard and clicks Approve or Reject.
4. `POST /api/pending/:id/approve` updates or inserts the alumni row and deletes the pending row; `DELETE /api/pending/:id` rejects it.
5. The frontend reloads the data and redraws the page.

5.4 Database Schema

Table	Columns	Notes
`alumni`	id, name, batch, dept, role, company, city, mentor (0/1), linkedin, bio, journey, achievements	`journey` and `achievements` are stored as JSON text
`talks`	id, speaker (alumni id), title, date, upcoming (0/1)	Foreign key to `alumni`
`achievements`	id, alum (alumni id), type, year, title, text	Foreign key to `alumni`
`pending`	pid, aid (alumni id or empty), name, changes	`changes` is JSON text; empty `aid` means a new member

5.5 Seeding

`server/db.js` creates the tables if they do not exist. When the `alumni` table is empty it loads `js/data.js` and inserts the 8 alumni, 4 talks, 4 achievements and 2 pending submissions in one transaction. Deleting `server/alumni.db` resets the data.

6. Setup and Installation Instructions

6.1 Prerequisites

- Node.js 22 or newer and npm.
- A modern web browser.

6.2 Run with the Database

1. Unzip `ysse-alumni-directory.zip` and open a terminal in the folder.
2. Install dependencies:
3. Start the server:
4. Open http://localhost:3000. The footer reads "Connected to database".

```
npm install

npm start
```

To use a different port, set the `PORT` environment variable before `npm start`.

6.3 Run in Demo Mode (No Server)

Open `index.html` directly in a browser. The footer reads "Demo mode (no server running)". The site uses the sample data in `js/data.js`, and changes reset when the page is refreshed.

6.4 Reset the Data

Stop the server, delete `server/alumni.db`, and start the server again. The database is recreated with the sample data.

7. Completed Features, Pending Work, and Issues Faced

7.1 Completed Features

1. Homepage with hero search, live alumni count and featured alumni.
2. Directory with text search and department, batch and mentoring filters.
3. Individual profile pages with journey timeline, achievements and a connection request form.
4. Alumni Talk page with upcoming and past sessions and alumni open to sharing.
5. Achievements page with success-story cards.
6. User dashboard with profile editing that goes to admin approval.
7. Admin dashboard with approve, reject and talk removal.
8. SQLite database, Express API and automatic seeding.
9. Demo mode that works without the server.
10. README and project documentation.

7.2 Pending / Partially Completed Work

1. **Login and roles:** the My profile page is fixed to one demo alumnus and the admin pages are open to anyone who can reach the server.
2. **Connection requests:** the form shows a confirmation but the request is not stored or delivered.
3. **Photos:** avatars are initials on a colored circle; there is no image upload.
4. **Talk links:** Join live session and Watch recording buttons are placeholders.
5. **Admin content management:** talks can be removed but not added or edited; achievements can not be managed yet.
6. **Automated tests:** none; verification was done with manual API calls (Section 8).

7.3 Issues Faced and Resolved

Ref	Problem	Cause	Resolution
BLD01	`js/data.js` was not created on the first build	The shell did not expand `{css,js}` and created one folder with that literal name	Removed the stray folder, created `css/` and `js/` separately and wrote the file again
BUG01	Page rendering line in `app.js` was invalid	A missing assignment operator in the `render()` function	Corrected to `app.innerHTML = ...`; syntax check passes
TST01	First API test did not prove approval worked	The server database started with an empty review queue, so the approve call used an ID that did not exist	Seeded the two pending submissions in `server/db.js` and repeated the full test (Appendix C)
DEP01	Install instructions named the wrong Node version	The documented minimum was 20, but better-sqlite3 requires 22	Updated the README and documentation to Node.js 22 or newer

8. Testing and Security Summary

8.1 Verification Performed

Check	Method	Result
Browser script syntax	`node` parse of `data.js`, `pages.js`, `app.js`	Passed
Load all data	`GET /api/data`	Returned alumni, talks, achievements and 2 pending submissions
Submit a change	`POST /api/pending` with an extra unknown field	Accepted; later approval succeeded, so the unknown field was dropped
Approve an update	`POST /api/pending/3/approve`	Alumnus 1 role changed to "Co-founder"

Check	Method	Result
Approve a new member	`POST /api/pending/1/approve`	Lamia Akter added; alumni count 8 to 9
Reject a submission	`DELETE /api/pending/2`	Removed from the queue; pending count 0
Remove a talk	`DELETE /api/talks/2`	Talk count 4 to 3
Database file exposure	`GET /server/alumni.db`	404 Not Found
Page review	Opening `index.html` in a browser (demo mode)	All pages render; see Section 4

Not covered: automated unit or integration tests, and browser testing of the pages against the running database server.

8.2 Security Measures in Place

- **Parameterized SQL:** every value reaches SQLite as a bound parameter; no SQL is built from user text.
- **Field whitelist:** only `name`, `batch`, `dept`, `role`, `company`, `city`, `mentor`, `linkedin`, `bio`, `journey`, `achievements` can be written, so column names never come from the request.
- **Output escaping:** all text inserted into pages passes through `esc()` in `pages.js`.
- **Request size limit:** JSON bodies are limited to 50 KB.
- **Static file scope:** the server serves only `index.html` and the `css` and `js` folders.
- **Contact privacy:** profiles show a LinkedIn link only when provided; connection requests go through the admin team.
- **Transactions:** approval changes and the pending-row deletion happen together.

8.3 Known Remaining Security Risks

1. **No authentication:** anyone who can reach the server can call the approve, reject and remove routes.
2. **No rate limiting** on any route.
3. **No dependency audit** has been run (`npm audit` was not executed).
4. **No HTTPS or security headers:** the server is meant for local use in its current form.

9. Known Limitations and Future Work

9.1 Known Limitations

- Data submitted in demo mode is kept in memory only and resets on refresh.
- SQLite allows one writer at a time, which is enough for this demo.
- Avatars use initials; sample data is fictional.

9.2 Future Roadmap

1. Login with alumni and admin roles, and protection of the admin routes.
2. Store and deliver connection requests.
3. Photo upload for profiles.
4. Admin screens to add and edit talks and achievements.
5. Real session links and recordings on the Alumni Talk page.
6. Pagination for a large directory and form validation messages.
7. Automated tests and deployment to a hosting service.

Appendix A: Configuration Reference

Setting	Required	Default	Description
`PORT` (environment variable)	No	3000	Port the Express server listens on
Database file	Automatic	`server/alumni.db`	Created on first run; delete it to reset the data
Request body limit	Fixed	50 KB	Maximum JSON body size accepted by the API

Appendix B: API Endpoint Reference

Method	Route Path	Description and Parameters
GET	`/`	Serves `index.html`
GET	`/css/*`, `/js/*`	Serves the stylesheet and scripts
GET	`/api/data`	Returns `{ alumni, talks, achievements, pending }`
POST	`/api/pending`	Stores a submission. Body: `{ aid, name, changes }`. `aid` is an alumni id or null for a new member; `name` is required; `changes` keeps only whitelisted fields. Returns 201, or 400 if `name` is missing
POST	`/api/pending/:id/approve`	Applies the submission to `alumni` and deletes it. Returns 404 if not found, 400 if a new member has no name
DELETE	`/api/pending/:id`	Rejects (deletes) a submission
DELETE	`/api/talks/:id`	Removes an Alumni Talk entry

Appendix C: Executed Verification Commands and Outputs

Run against the server started on port 3111 with a fresh database:

```
pending at start:      "pid":1 "pid":2
POST /api/pending            -> {"ok":true}
POST /api/pending/3/approve  -> {"ok":true}
POST /api/pending/1/approve  -> {"ok":true}
DELETE /api/pending/2        -> {"ok":true}
DELETE /api/talks/2          -> {"ok":true}
GET /api/data summary        -> alumni 9 | id1 role: Co-founder |
                                last: Lamia Akter | talks 3 | pending 0
GET /server/alumni.db        -> 404
```